An efficient primal-dual Newton method for projections onto $\ell_{1,\infty}$ norm ball

Song Mao [*1] **Hao Wang** [*1]

Abstract

We consider the projection onto the $\ell_{1,\infty}$ norm ball for solving group sparse optimization problems arising in multi-task learning. Based on the primal-dual gradient method (PDG), we present a novel primal-dual Newton method, which updates the primal and dual iterates incrementally with Newton steps. We exploit the problem-inherent structure so that the closed-form for the primal-dual Newton steps can be derived easily. Compared with existing algorithms, our approach does not need to maintain the primal feasibility, nor require the computation of the ℓ_1 ball projection subproblems. We prove that our algorithm terminates after a finite number of iterations. Numerical simulations on synthetic and real-world data show that our proposed method is faster than the previous state-of-the-art.

1. Introduction

1.1. Background

Optimization problems involve group-sparse appear in a variety of applications and are popular in engineering, statistics and many other fields. For example, it is widely applied in machine learning for improving the stability of the feature selection (Sra, 2011; 2012) and solving collective face recognition problems (Wang & Chen, 2017), resulting in various mixed-norm constrained regression methods (Kowalski, 2009). It is also used for signal denoising (Kowalski & Torrésani, 2009) and simultaneous sparse signal approximation (Tropp, 2006) in signal processing. To promote group-sparse structure in the variables of interest, the $\ell_{1,p}$ norm ($p \geq 2$) constraint has been shown to be useful. Generally, large p indicates strong correlation among the groups of the variables (Obozinski et al., 2010). In particular, the $\ell_{1,\infty}$ norm ball constrained optimization problems have attracted numerous attentions in many applications (Béjar et al., 2019; Gustavo et al., 2018; Chau et al., 2019) such as image annotation (Quattoni et al., 2009), multi-task learning (Liu et al., 2009) and multitask lasso regression (Sra, 2011; 2012).

In this paper, we consider solving the $\ell_{1,\infty}$ norm ball constrained optimization problems of the form:

$$\begin{aligned} \min_{\mathbf{W}\in\mathbb{R}^{m\times n}} \quad & f(\mathbf{W}) \\ \text{s.t.} \quad & \|\mathbf{W}\|_{1,\infty} \leq C, \end{aligned} \tag{1}$$

where $f : \mathbb{R}^{m\times n} \to \mathbb{R}$ is a convex differentiable loss functions measures the error of $\mathbf{W}$. $C > 0$ is a constant that controls the group sparsity which can be user-specified. The other commonly seen formulation is to penalize the $\ell_{1,\infty}$ norm of $\mathbf{W}$ into the objective. Compared with the penalized version, the constrained version here (1) is usually more favorable since after selecting the value of C, one can know the group sparsity of the solutions prior to solving the problem.

One of the most popular methods for handling convex constraint optimization problems is the gradient projection algorithm (GPA) (Rosen, 1960; Birgin et al., 2000), which is often considered as a scalable solver for large-scale optimization problems, the update of the GPA applied to (1) is defined as

$$\mathbf{W}^{k+1} \leftarrow \mathcal{P}_{\ell_{1,\infty},C}\left(\mathbf{W}^k - \gamma^k \nabla f(\mathbf{W}^k)\right), \tag{2}$$

where $\gamma^k > 0$ is the stepsize and $\mathcal{P}_{\ell_{1,\infty},C}(\cdot)$ is the Euclidean projection onto the $\ell_{1,\infty}$ ball with radius C that will be defined in the next section formally.

GPA enjoys fast convergence rate (Bertsekas, 1976) compared with many other first-order projection-free methods such as the Frank-Wolfe method (Frank et al., 1956; Luss & Teboulle, 2013) (*a.k.a.* conditional gradient method), and can incorporate many techniques such as the spectral step size (Barzilai & Borwein, 1988) and non-monotone line search (Grippo et al., 1986) to improve the performance. At each iteration, GPA moves along the negative gradient and then projects the results point to satisfy the constraints as stated in (2). Therefore, its performance critically depends on the efficiency of projection operation and may suffer

[*]Equal contribution [1]School of Information Science and Technology, ShanghaiTech University, Shanghai, China. Correspondence to: Mao Song <maosong@shanghaitech.edu.cn>, Wang Hao <wanghao1@shanghaitech.edu.cn>.

Proceedings of the 39^{th} International Conference on Machine Learning, Baltimore, Maryland, USA, PMLR 162, 2022.

from heavy computational burden if the projection subproblem is difficult to solve. Therefore, our primary focus in this paper is on designing a fast projection method onto the $\ell_{1,\infty}$ norm ball.

1.2. Related work

To find the projection onto the $\ell_{1,\infty}$ norm ball, several methods have appeared in the past decade. A common approach is to reformulate the dual problem of the projection as a univariate equation, and then using a root-finding technique to determine the optimal dual solution. (Quattoni et al., 2009) proposed a sorting-based algorithm (SBM) to find the root of a piece-wise linear function with $mn+1$ breakpoints, which is shown to have the worst case complexity of $\mathcal{O}(mn\log(mn))$. (Sra, 2011) studied the projection onto the $\ell_{1,q}$ $(q>1)$ norm ball and converted the projection problem to a root-finding problem, which can then be attacked by the bisection method.

The dual method was then outperformed by the primal-dual methods, for example, the dual ascent (DA) method. DA constructs the Lagrangian of the projection problem; at each iteration, it finds the exact solution to the primal subproblem for a fixed dual variable and then updates the dual variable by a gradient ascent step. (Gustavo et al., 2018) first uses DA to solve the $\ell_{1,\infty}$ norm ball projection problem by updating the dual variable with the Steffensen root search method. (Chau et al., 2019) then improved the Steffensen method with the Newton root search method (NRFM), which achieves the best performance among all root-finding-based methods. (Béjar et al., 2019) proposed a general iterative algorithm for computing the proximal operator of the $\ell_{1,\infty}$ norm and proved that the algorithm converges in finite steps.

The DA relies on computing the optimal solution to the ℓ_1 norm ball projection subproblem, which may become a computational burden if the dimension becomes large. The most recent work (Chu et al., 2020) proposed a semi-smooth Newton algorithm (SNA) by solving the KKT equations directly. The use of the second-order information enables it to become the state-of-the-art solver. Moreover, a local Q-superlinearly convergence rate or the finite step convergence of SNA is established.

1.3. Contribution

In this paper, we proposed a primal-dual Newton method for solving the $\ell_{1,\infty}$ norm ball projection, which is based on the well-known primal-dual gradient (PDG) method for solving convex constrained optimization problems. Different from DA, the PDG updates the primal variable by gradient descent steps and the dual variable by gradient ascent steps incrementally. Compared with the DA, the PDG can avoid repeatedly solving multiple ℓ_1 norm ball projection subproblems in our case, and attains a linear convergence rate since the problem is strongly convex and strongly smooth. (Du & Hu, 2019; Alghunaim & Sayed, 2020).

The difference between our method and PDG is that we update the primal and dual variables all by the Newton steps. Usually, the computational cost for the Newton steps is not trivially, since it needs to evaluate the second-order derivative and then solve the Newton equation. However, in our case, we can exploit the problem-inherent structure to derive the closed-form solution of the Newton steps. We show that this method terminates in finite steps — a novel feature that the general PDG does not possess.

Moreover, our proposed algorithm is more robust to initialization of the dual variable compared to SNA, since SNA has the restrictions on the primal feasibility, and an improper initial guess may causes divergence of the algorithm. Table 1 gives a qualitative comparison of existing methods for computing the projection onto the $\ell_{1,\infty}$ norm ball with respect to different algorithmic properties.

1.4. Organization

The remainder of this paper is outlined as follows. In Section 2, we state the problem formulation and the optimality conditions. In Section 3, we exploit the property of the projection and describe the proposed algorithm in detail. We provide the convergence analysis in Section 4. Numerical experiments are presented in Section 5. Concluding remarks are provided in Section 6.

1.5. Notation

Let $\mathbb{R}^n$ be the n-dimensional Euclidean space and $\mathbb{R}_+$ be the nonnegative orthant. For any $\boldsymbol{x}\in\mathbb{R}^n$ and $p\geq 1$, we denote the ℓ_p norm as $\|\boldsymbol{x}\|_p=(\sum_{i=1}^n|x_i|^p)^{1/p}$; the infinity norm is defined as $\|\boldsymbol{x}\|_\infty=\max\{|x_1|,\ldots,|x_n|\}$. We use $[x]_+=\max(x,0)$ and denote $[n]=\{1,\ldots,n\}$ for simplicity. We represent a matrix $\mathbf{A}=(A_{ij})\in\mathbb{R}^{m\times n}$ with its rows: $\mathbf{A}=[\boldsymbol{a}_1;\ldots;\boldsymbol{a}_m]$, where $\boldsymbol{a}_i\in\mathbb{R}^{1\times n}$ is the i-th row of $\mathbf{A}$, so $\boldsymbol{a}_{ij}=A_{ij}$, we will use two kinds of representation alternatively in this paper for convenience. For any $\mathbf{A},\mathbf{B}\in\mathbb{R}^{m\times n}$, the absolute value of $\mathbf{A}$ is define as $|\mathbf{A}|=(|A_{ij}|)$, the Hadamard product between $\mathbf{A}$ and $\mathbf{B}$ is defined as $\mathbf{A}\odot\mathbf{B}=(A_{ij}B_{ij})$. We also define $\mathcal{I}(\mu_i)$ as the index of elements of i-th row of $\mathbf{A}$ that is greater than or equal to μ_i(Chu et al., 2020), that is, $\mathcal{I}(\mu_i)=\{j|A_{ij}\geq\mu_i\}$. Let $|S|$ be the cardinality of a given index set S. The Frobenuis norm of $\mathbf{A}$ is given by $\|\mathbf{A}\|_F=\left(\sum_{i=1}^m\sum_{j=1}^n A_{ij}^2\right)^{1/2}$, and the $\ell_{1,\infty}$ norm of $\mathbf{A}$ is defined as

$$\|\mathbf{A}\|_{1,\infty}=\|[\|\boldsymbol{a}_1\|_\infty;\ldots;\|\boldsymbol{a}_m\|_\infty]\|_1=\sum_{i=1}^m\|\boldsymbol{a}_i\|_\infty. \quad (3)$$

Table 1. Qualitative comparison of different methods

METHOD	FINITE STEP	SUBPROBLEM SOLVING	FEASIBILITY	PRIMAL/DUAL ORDER
DA (BÉJAR ET AL., 2019)	YES	YES	NO	FIRST/SECOND
IPM (CHU ET AL., 2020)	YES	NO	YES	SECOND/SECOND
PDGM (ALGHUNAIM & SAYED, 2020)	NO	NO	NO	FIRST/FIRST
PDNM (PROPOSED)	YES	NO	NO	SECOND/SECOND

It can be seen that the $\ell_{1,\infty}$ norm is determined by the largest absolute value of each row. If $\|\boldsymbol{a}_i\|_\infty = 0$, the corresponding row of $\mathbf{A}$ is also zero. If $f : \mathbb{R}^n \to \mathbb{R}$ is a convex function, then the subdifferential of f at point $\bar{\boldsymbol{x}}$ is given by $\partial f(\bar{\boldsymbol{x}}) := \{\boldsymbol{z} \mid f(\bar{\boldsymbol{x}}) + \langle \boldsymbol{z}, \boldsymbol{x} - \bar{\boldsymbol{x}} \rangle \leq f(\boldsymbol{x}), \forall\, \boldsymbol{x} \in \mathbb{R}^n\}$. An element in $\partial f(\bar{\boldsymbol{x}})$ is called the subgradient of f at $\bar{\boldsymbol{x}}$. In particular, if f is differentiable, then $\partial f(\bar{\boldsymbol{x}}) = \{\nabla f(\bar{\boldsymbol{x}})\}$.

2. Problem Formulation

Given a matrix $\mathbf{A} \in \mathbb{R}^{m\times n}$, we focus on computing the projection of $\mathbf{A}$ onto the $\ell_{1,\infty}$ ball:

$$\begin{aligned} \min_{\mathbf{W}\in\mathbb{R}^{m\times n}} \quad & \frac{1}{2}\|\mathbf{W} - \mathbf{A}\|_F^2 \\ \text{s.t.} \quad & \|\mathbf{W}\|_{1,\infty} \leq C, \end{aligned} \tag{4}$$

where $C > 0$ is the radius of the $\ell_{1,\infty}$ ball.

To simplify problem (4), we first introduce the following lemma, which restricts the sign of the entries of $\mathbf{A}$ to be non-negative, the proof is omitted here and we refer to (Béjar et al., 2019; Duchi et al., 2008) for interested readers.

Lemma 1 (Matched sign)**.** *The sign of the optimal solution $\mathbf{W}^*$ of problem (4) must match the sign of $\mathbf{A}$, that is, $W_{ij}^* A_{ij} \geq 0, \forall (i,j) \in [m] \times [n]$.*

We can now assume that $\mathbf{A} \in \mathbb{R}_+^{m\times n}$ since otherwise by Lemma 1 the optimal solution can be obtained by first computing the projection of $|\mathbf{A}|$ onto the $\ell_{1,\infty}$ ball and then multiplying it element-wise by $sign(\mathbf{A})$. Moreover, note that if $\|\mathbf{A}\|_{1,\infty} \leq C$, then the solution to (4) is simply $\mathbf{A}$ itself, so we also assume that $\|\mathbf{A}\|_{1,\infty} > C$ in this paper.

The dimension of problem (4) is mn, which can be reduced to m. Using the property of $\ell_{1,\infty}$ norm, we first introduce the slack variable $\mu_i = \|\boldsymbol{w}_i\|_\infty$ for $i = 1, \ldots, m$, and rewrite problem (4) as

$$\begin{aligned} \min_{\mathbf{W},\boldsymbol{\mu}} \quad & \frac{1}{2}\sum_{i=1}^m \|\boldsymbol{w}_i - \boldsymbol{a}_i\|_2^2 \\ \text{s.t.} \quad & \sum_{i=1}^m \mu_i = C \\ & 0 \leq W_{ij} \leq \mu_i, \forall (i,j) \in [m] \times [n] \end{aligned} \tag{5}$$

If we fix $\boldsymbol{\mu}$, the problem above is separable to $\boldsymbol{w}_i$:

$$\begin{aligned} \min_{\boldsymbol{w}_i} \quad & \frac{1}{2}\|\boldsymbol{w}_i - \boldsymbol{a}_i\|_2^2 \\ \text{s.t.} \quad & \mathbf{0}_{1\times n} \preceq \boldsymbol{w}_i \preceq \mu_i \mathbf{1}_{1\times n}, \end{aligned} \quad , \; i = 1, \ldots, m, \tag{6}$$

where $\mathbf{a} \preceq \boldsymbol{b}$ means $\boldsymbol{b} - \mathbf{a} \in \mathbb{R}_+$. Note that problem (6) is the projection of $\boldsymbol{a}_i$ onto the box constraint, which has a closed-form solution (Beck, 2017):

$$\boldsymbol{w}_i = \min(\boldsymbol{a}_i, \mu_i \mathbf{1}_{1\times n}). \tag{7}$$

Substituting (7) back into problem (5), we obtain an equivalent optimization problem with dimension m:

$$\begin{aligned} \min_{\boldsymbol{\mu}\in\mathbb{R}^m} \; & \frac{1}{2}\sum_{i=1}^m \sum_{j=1}^n [A_{ij} - \mu_i]_+^2 \\ \text{s.t.} \; & \sum_{i=1}^m \mu_i = C \\ & \mu_i \geq 0, i = 1, \ldots, m \end{aligned} \quad . \tag{8}$$

Now we construct the Lagrangian of problem (8):

$$\mathcal{L}(\boldsymbol{\mu}, \lambda) = \frac{1}{2}\sum_{i=1}^m \sum_{j=1}^n [A_{ij} - \mu_i]_+^2 + \lambda \left(\sum_{i=1}^m \mu_i - C \right). \tag{9}$$

By strong duality, problem (8) can be reformulated as the following saddle point problem (Bauschke & Combettes, 2011):

$$\min_{\boldsymbol{\mu}\in\mathbb{R}_+^n} \max_{\lambda\in\mathbb{R}} \mathcal{L}(\boldsymbol{\mu}, \lambda). \tag{10}$$

The saddle point of (10), which we aim to solve for, is defined as follows.

Definition 1. *A point $(\boldsymbol{\mu}^*, \lambda^*) \in \mathbb{R}_+^m \times \mathbb{R}$ is a* saddle point *of problem (10) if*

$$\mathcal{L}(\boldsymbol{\mu}^*, \lambda) \leq \mathcal{L}(\boldsymbol{\mu}^*, \lambda^*) \leq \mathcal{L}(\boldsymbol{\mu}, \lambda^*), \forall (\boldsymbol{\mu}, \lambda) \in \mathbb{R}_+^m \times \mathbb{R}.$$

If $(\boldsymbol{\mu}^*, \lambda^*)$ is a saddle point of problem (10), then $\boldsymbol{\mu}^*$ is the optimal solution to problem (8) (Rockafellar, 2015). We first introduce the optimality conditions for saddle point problem (10):

Lemma 2. *The saddle point $(\boldsymbol{\mu}^*, \lambda^*)$ of problem (10) satisfies the following optimality conditions:*

$$\nabla_i \mathcal{L}(\boldsymbol{\mu}^*, \lambda^*) := -\sum_{j=1}^{n} [A_{ij} - \mu_i^*]_+ + \lambda^* = 0, \quad (11)$$

$$\nabla_\lambda \mathcal{L}(\boldsymbol{\mu}^*, \lambda^*) := \sum_{i \in \mathcal{B}^*} \mu_i^* - C = 0, \quad (12)$$

$$\mu_i^* \geq 0, i \in [m]. \quad (13)$$

Now our goal is to solve problem (10). Once the saddle point $(\boldsymbol{\mu}^*, \lambda^*)$ is found, the following lemma can be used to retrieve the optimal solution to problem (4).

Lemma 3. *Let $(\boldsymbol{\mu}^*, \lambda^*)$ be the saddle point to problem (10). Then the optimal solution $\mathbf{W}^*$ to (4) satisfies the following equation:*

$$\boldsymbol{w}_i^* = \begin{cases} \min(\boldsymbol{a}_i, \mu_i^* \mathbf{1}_{1\times n}), & \text{if } i \in \mathcal{B}^* \\ \mathbf{0}_{1\times n}, & \text{otherwise} \end{cases}, \quad (14)$$

where $\mathcal{B}^* := \{i \mid \|\boldsymbol{a}_i\|_1 > \lambda^*\}$.

3. Algorithm

We first analyze the structure of the problem from the primal and the dual perspective in Section 3.1 and 3.2. Then, we present our proposed algorithm in Section 3.3.

3.1. Primal update

We first define $\mu_i(\lambda^k)$ as the exact solution of the primal update at the k-th iteration for given λ^k, meaning it satisfies

$$-\sum_{j=1}^{n} \left[A_{ij} - \mu_i(\lambda^k)\right]_+ + \lambda^k = 0. \quad (15)$$

It be seen from the optimal condition (11) and the fact that $\nabla_i \mathcal{L}(\boldsymbol{\mu}, \lambda)$ is a monotone function of μ_i, we have $\mu_i^* = \mu_i(\lambda^*)$.

Due to the fact that $\nabla_i \mathcal{L}(\boldsymbol{\mu}^k, \lambda^k)$ is a piece-wise linear function with respect to μ_i with at most n breakpoints, inspired by (Rodriguez, 2017; Michelot, 1986), the primal update in this paper takes the form:

$$\mu_i^{k+1} = \mu_i^k - \beta_i^k \nabla_i \mathcal{L}(\boldsymbol{\mu}^k, \lambda^k), \ i \in \mathcal{B}^k, \quad (16)$$

where $\beta_i^k = 1/|\mathcal{I}(\mu_i^k)|$ and $\mathcal{B}(\lambda) := \{i \mid \|\boldsymbol{a}_i\|_1 > \lambda\}$. We also use $\mathcal{B}^k$ as shorthand for $\mathcal{B}(\lambda^k)$. Since $\nabla_i \mathcal{L}(\boldsymbol{\mu}^k, \lambda^k)$ is convex with respect to μ_i and $-1/\beta_i^k \in \partial \nabla_i \mathcal{L}(\boldsymbol{\mu}^k, \lambda^k)$, (16) is therefore a Newton update, meaning that the next iterate μ_i^{k+1} is determined by the intersection of the tangent line of $\nabla_i \mathcal{L}(\boldsymbol{\mu}^k, \lambda^k)$ at the point $\boldsymbol{\mu}^k$ and the μ axis. We depict this updating strategy in Fig 1. If $\mu_i^k \leq \|\boldsymbol{a}_i\|_\infty$ and $\lambda^k > 0$, then the update (16) is well-defined.

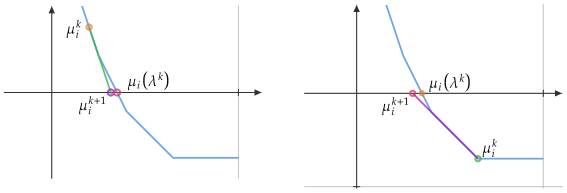

Figure 1. Primal update scheme

Notice that in update (16), it does not necessarily hold $\mu_i^{k+1} = \mu_i(\lambda^k)$ for a given λ^k. We do not use $\mu_i(\lambda^k)$ as our next iteration, since it needs to solve the piece-wise linear equation (15), which has a worst complexity of $O(n^2)$ (Michelot, 1986). This represents the main difference from DA, since DA involves computing $\mu_i(\lambda^k)$ at each iteration for a given λ^k. We summarize this in the following lemma.

Lemma 4. *For $i \in [m]$, $\mu_i^k \leq \|\boldsymbol{a}_i\|_\infty$. If $\beta_i^k = 1/|\mathcal{I}(\mu_i^k)|$, then $\mu_i^{k+1} \leq \mu_i(\lambda^k)$. Moreover, if $\lambda^{k+n+1} = \cdots = \lambda^k$, meaning λ is unchanged over iterations $k, \ldots, k+n+1$, then there exists $j \in [n]$ such that $\mu_i^{k+n+1} = \cdots = \mu_i^{k+j} = \mu_i(\lambda^k)$.*

Proof. Let $h(\mu) := \sum_{i=1}^m [A_{ij} - \mu]_+ - \lambda^k$, which is a convex, piece-wise linear and non-increasing function. Let $\pi(\mu) := h(\mu_i^k) + h'(\mu_i^k)(\mu - \mu_i^k)$, where $h'(\mu_i^k) = -|\mathcal{I}(\mu_i^k)| \in \partial h(\mu_i^k)$. It holds that $h(\mu_i^{k+1}) \geq \pi(\mu_i^{k+1}) = 0$. Therefore, $\mu_i^{k+1} \leq \mu_i(\lambda^k)$ by the fact that $h(\mu)$ is monotonically non-increasing.

Let $\mu_i^k \in [\theta_1, \theta_2)$, where $\theta_1 < \theta_2$ are two consecutive breakpoints of $h(\mu)$. Notice that h and π have the same slope $h'(\mu_i^k)$ on $[\theta_1, \theta_2)$ and $h(\mu_i^k) = \pi(\mu_i^k)$, hence $\pi(\mu) = h(\mu)$ for $\mu \in [\theta_1, \theta_2)$ If $\mu_i(\lambda^k) \in [\theta_1, \theta_2]$, we have $\pi(\mu(\lambda^k)) = 0$ and $\mu_i^{k+1} = \mu(\lambda^k)$; if $\mu_i(\lambda^k) > \theta_2$, we have $\pi(\theta_2) = h(\theta_2) > 0$. So $\mu_i^{k+1} > \theta_2$. Since there are at most $n+1$ breakpoints, we must have $\mu_i^{k+n+1} = \cdots = \mu_i^{k+j} = \mu_i(\lambda^k)$ for some $j \in [n]$. □

The above Lemma reveals that μ_i^{k+1} is always an underestimate of $\mu_i(\lambda^k)$. In the worst case, if the dual variable remains unchanged, then we obtain $\mu_i(\lambda^k)$ in at most n steps. If this happens, then our algorithm behaves the same as NRFM proposed by (Chau et al., 2019) for finding the new iterate μ_i^{k+1}.

3.2. Dual update

The update of the dual variable is given by:

$$\lambda^{k+1} = \lambda^k + \alpha_k \nabla_\lambda \mathcal{L}(\boldsymbol{\mu}^{k+1}, \lambda^k). \quad (17)$$

This dual update is generally adopted by primal dual gradient method for equality constraint optimization. Typically, α_k is chosen as some constant related to the strongly convex parameter (Du & Hu, 2019; Alghunaim & Sayed, 2020). To achieve faster convergence, after obtaining $\boldsymbol{\mu}^{k+1}$, we do not adopt this kind of updating strategy for λ. Since $\nabla_\lambda \mathcal{L}(\boldsymbol{\mu}^{k+1}, \lambda)$ is Lipschitz continuous with respect to λ, $\nabla_{\lambda\lambda} \mathcal{L}(\boldsymbol{\mu}^k, \lambda)$ exists almost everywhere (Federer, 2014), thus we can invoke semismooth Newton methods to update λ^k , which takes the following form:

$$\nabla_{\lambda\lambda} \mathcal{L}(\boldsymbol{\mu}^{k+1}, \lambda^k)(\lambda^{k+1} - \lambda^k) - \nabla_\lambda \mathcal{L}(\boldsymbol{\mu}^{k+1}, \lambda^k) = 0, \tag{18}$$

where $\nabla_{\lambda\lambda} \mathcal{L}(\boldsymbol{\mu}^k, \lambda^k)$ is a general Hessian (Klintberg & Gros, 2014). Note that $\nabla_\lambda \mathcal{L}(\boldsymbol{\mu}^{k+1}, \lambda^k) = \sum_{i\in\mathcal{B}^k} \mu_i^{k+1} - C$, we take the derivative of μ_i^{k+1} with respect to λ^k and obtain the Newton step for obtaining λ^{k+1}:

$$\lambda^{k+1} = \lambda^k + \frac{\left[\sum_{i\in\mathcal{B}^k} \mu_i^{k+1} - C\right]_+}{\sum_{i\in\mathcal{B}^k} \frac{1}{|\mathcal{I}(\mu_i^{k+1})|}}, \tag{19}$$

which means a Newton step can be obtained by setting $\alpha^k = 1/\sum_{i\in\mathcal{B}^k} \beta_i^{k+1}$. By the definition of $\mathcal{B}^k$, $\mathcal{I}(\mu_i^{k+1}) \neq \emptyset$ always holds, so that the stepsize α^k is always and well-defined. Compared with the original update, we add $[\cdot]_+$ to create a monotonically increasing dual variable sequence.

3.3. Proposed algorithm

Our proposed algorithm , an alternating primal-dual Newton (PDN) method can now be summarized in Algorithm 1.

Though we set the initialization in the Algorithm 1 as $\boldsymbol{\mu}^0 = \mathbf{0}_{m\times 1}$. It should be emphasized that our initialization of $\boldsymbol{\mu}^0$ does not depend on λ^0 as SNA, which requires that $\mu_i^0 = \mu_i(\lambda^0)$ or it will suffer from the slowly convergence rate even the diverge problems.

Moreover, we can see that the memory space needed by our algorithm is less than that of SNA, since we only have to save $\boldsymbol{\mu}^k$, whereas sna needs to save $\boldsymbol{\mu}^k$ and $\boldsymbol{\mu}^{k+1}$ simultaneously. our algorithm is capable of saving the memory space by updating $\boldsymbol{\mu}^k$ in place. This is important in large-scale problems since stores $\boldsymbol{\mu}^k$ may be also expensive.

4. Convergence Analysis

Let $s(\lambda) := \sum_{i\in\mathcal{B}^k} \mu_i(\lambda) - C$ where $\mu_i(\lambda)$ is defined in (15). It can be seen that $s(\lambda)$ is a monotone decreasing function with $s'(\lambda) := \sum_{i\in\mathcal{B}^k} 1/|\mathcal{I}(\mu_i(\lambda))| \in \partial s(\lambda)$. We will use this property in the proof of convergence analysis. The range of the optimal dual variable λ^* can be determined as follows.

Lemma 5. *The optimal dual variable λ^* defined in Lemma 2 satisfies $\lambda^* \in [\lambda_l, \lambda_r)$, where $\lambda_l = (\sum_{i=1}^m \|\boldsymbol{a}_i\|_1 - C)/m$ and $\lambda_r = \max_i \|\boldsymbol{a}_i\|_1$.*

Algorithm 1 Primal-dual Newton Method

Input: $\mathbf{A} \in \mathbb{R}_+^{m\times n}$, $C > 0$.
if $\|\mathbf{A}\|_{1,\infty} \leq C$ **then**
  return $\mathbf{A}$
end if
Initialize $\lambda^0 = (\sum_{i=1}^m \|\boldsymbol{a}_i\|_\infty - C)/m$, $\boldsymbol{\mu}^0 = \mathbf{0}_{m\times 1}$ and $\beta_i^0 = 1/|\mathcal{I}(\mu_i^0)|$ for $i \in [m]$.
for $k = 0, 1, 2, \ldots$ **do**
  Update $\mathcal{B}^k = \{i | \|\boldsymbol{a}_i\|_1 > \lambda^k\}$.
  Update μ_i^k

$$\mu_i^{k+1} = \begin{cases} \mu_i^k - \beta_i^k \nabla_i \mathcal{L}\left(\boldsymbol{\mu}^k, \lambda^k\right), & \text{if } i \in \mathcal{B}^k \\ 0, & \text{otherwise} \end{cases} \tag{20}$$

  Compute $\beta_i^{k+1} = 1/|\mathcal{I}(\mu_i^{k+1})|$ for $i \in \mathcal{B}^k$.
  Update λ^k

$$\lambda^{k+1} = \lambda^k + \frac{\left[\sum_{i\in\mathcal{B}^k} \mu_i^{k+1} - C\right]_+}{\sum_{i\in\mathcal{B}^k} \beta_i^{k+1}}. \tag{21}$$

  if $|\sum_{i\in\mathcal{B}^k} \mu_i^{k+1} - C| \leq 0$ **then**
    Break
  end if
end for
return $\mathbf{W}^* = \min(\mathbf{A}, \boldsymbol{\mu}^k \mathbf{1}_{1\times n})$.

Proof. By Lemma 2, we have $\mathcal{B}(\lambda_r) = \{i \mid \|\boldsymbol{a}_i\|_1 > \lambda_r\} = \emptyset$, so $s(\lambda_r) = \sum_{i\in\mathcal{B}^r} \mu_i(\lambda_r) - C = -C < 0$ and $\lambda^* < \lambda_r$. On the other hand, we can rewrite (11) to have

$$\begin{aligned} \lambda^* &= \frac{1}{m}\left[\sum_{i\in\mathcal{B}^*} [A_{ij} - \mu_i^*]_+ + \sum_{i\notin\mathcal{B}^*} [A_{ij} - 0]_+\right] \\ &= \frac{1}{m}\sum_{i=1}^m [A_{ij} - \mu_i^*]_+ \geq \frac{1}{m}\sum_{i=1}^m (A_{ij} - \mu_i^*) \\ &= \frac{1}{m}\left[\sum_{i=1}^m \|\boldsymbol{a}_i\|_1 - \sum_{i=1}^m \mu_i^*\right] = \lambda_l. \end{aligned} \tag{22}$$

This completes the proof. □

We denote $\{\lambda^k\}$ as the sequence generated by Algorithm 1 in this section. The next result provides the main property of $\{\lambda^k\}$.

Lemma 6. *$\{\lambda^k\}$ is monotonically non-decreasing and converges. Moreover, $\lim_{k\to\infty} \lambda^k \leq \lambda^*$.*

Proof. We prove this conclusion by induction, $\lambda^0 \leq \lambda^*$ by Lemma 5, we now prove that if $\lambda^k \leq \lambda^*$, then $\lambda^{k+1} \leq \lambda^*$. First of all, this is trivially true if $\sum_{i\in\mathcal{B}^k} \mu_i^{k+1} - C \leq 0$ by the update of (19). Now we consider the case that

$\sum_{i\in\mathcal{B}^k}\mu_i^{k+1} - C > 0$. It follows from Lemma 4 that $\sum_{i\in\mathcal{B}^k}\mu_i^{k+1} - C \le \sum_{i\in\mathcal{B}^k}\mu_i(\lambda^k) - C = s(\lambda^k)$, then the update of λ^k implies

$$\lambda^{k+1} \le \lambda^k + \frac{s(\lambda^k)}{\sum_{i\in\mathcal{B}^k}\beta_i^{k+1}} \le \tilde{\lambda}^{k+1} := \lambda^k + \frac{s(\lambda^k)}{s'(\lambda^k)}, \quad (23)$$

where $s'(\lambda^k) \in \partial s(\lambda^k)$. Similar to the proof of Lemma 4, it holds that $s(\tilde{\lambda}^{k+1}) \ge s(\lambda^k) + s'(\lambda^k)(\tilde{\lambda}^{k+1} - \lambda^k) = 0$, therefore $\tilde{\lambda}^{k+1} \le \lambda^*$ by the fact that $s(\lambda)$ is monotonically decreasing, yielding $\lambda^{k+1} \le \lambda^*$ by (23). By induction, $\{\lambda^k\}$ is bounded above by λ^*. Therefore $\{\lambda^k\}$ converges and $\lim_{k\to\infty}\lambda^k \le \lambda^*$. □

Theorem 1. *The sequence $\{\lambda^k\}$ converges to λ^*. That is, $\lim_{k\to\infty}\lambda^k = \lambda^*$.*

Proof. We prove this statement by contradiction. We suppose that there exists $\sigma > 0$ such that $\bar{\lambda} := \lim_{k\to\infty}\lambda^k < \lambda^* - \sigma$. Since $s(\lambda)$ is a continuous linear function, $s(\bar{\lambda}) > L\epsilon$ for some $L > 0$. However, note that we have $\lambda^k = \bar{\lambda}$ in (23) by Lemma 4 ($\lambda^j = \bar{\lambda}$ for $j = k+1, \dots, ...$). However, note that $\alpha^k \ge 1/m$, we have $\bar{\lambda} + s(\bar{\lambda})/\alpha^k > \bar{\lambda} + L\sigma/m$, which contradicts with the fact that $\bar{\lambda}$ is the limit of $\{\lambda^k\}$. Therefore, we have $\lim_{k\to\infty}\lambda^k = \lambda^*$. □

A direct consequence of Theorem 1 and Lemma 4 is that our proposed algorithm produces a sequence that converges to the optimal primal-dual pair $(\boldsymbol{\mu}^*, \lambda^*)$.

Corollary 1. *The sequence $(\boldsymbol{\mu}^k, \lambda^k)$ generated by Algorithm 1 converges to the KKT point of problem (8).*

We introduce the following lemma which illustrates the structure of problem (10) from the dual perspective, the proof can be found in (Frasch et al., 2013; Joachim Ferreau et al., 2012).

Lemma 7. *Let $q(\lambda) = \min_{\boldsymbol{\mu}} \mathcal{L}(\boldsymbol{\mu}, \lambda)$, then $q(\lambda)$ is a piecewise-quadratic, concave and first order differentiable function.*

Theorem 2. *The Algorithm 1 terminates in finite steps.*

Proof. We have proved that $\{\lambda^k\}$ converges to λ^*. From Lemma 7, the dual objective $q(\lambda)$ is a piecewise-quadratic function. When $|\lambda^k - \lambda^*| < \sigma$ for sufficiently small $\sigma > 0$, meaning that λ^k and λ^* lie in the same interval. Then it can be seen from the proof Lemma 4 that $\boldsymbol{\mu}^{k+1} = \boldsymbol{\mu}^*$. Then equation holds in (23) and the update is exactly the Newton step at λ^k. $q(\lambda)$ is a quadratic function in this interval means the Newton step finds the optimum in one step, that is, $\lambda^{k+1} = \lambda^*$. This proves the finite step convergence. □

Though we do not give the explicit number of iteration of our algorithm as (Chu et al., 2020; Béjar et al., 2019). Our algorithm will terminates in at most n^2m itrations in worst case. This is because if we fix λ^k and solves $\mu_i(\lambda^k)$ exactly (which takes at most n steps from Lemma 4), then our algorithm reduces to Algorithm 4 of (Béjar et al., 2019) and converges in at most mn iterations.

5. Experiment Results

In this section, we validate the performance of our proposed algorithm. All experiments are done in a 2.40GHz, 8 core precision tower with Intel E5 processor and using MATLAB2021b. We denote our algorithm as PDN and the state-of-the-art algorithm proposed by (Chu et al., 2020) SNA[1], the benchmark SBM(Quattoni et al., 2009)[2] is also the same as used in (Chu et al., 2020). SBM, SNA are re-implemented with MATLAB for consistency (code of SBM written by the author is used directly, SNA and NRFM are re-implemented by MATLAB for fairness). Our implementation will be released publicly on the Github for reproducing the results.

5.1. Performance of proposed algorithm

Similar to (Chu et al., 2020), we first choose various factor $\gamma \in [0, 1]$ such that the radius of the $\ell_{1,\infty}$ norm ball is given by $C = \gamma\|\mathbf{A}\|_{1,\infty}$. SBM, SNA, PDN are then invoked to solve problem (4), while NRFM(Chau et al., 2019)[3] is not used for speed comparison since its performance was reported in (Chu et al., 2020) to be outperformed by SNA. The CPU time is recorded for each algorithm and all experiment results are the average of 10 trials. The terminal criterion for PDN is set as

$$\left|\sum_{i\in\mathcal{B}^k}\mu_i^{k+1} - C\right| \le \epsilon, \quad (24)$$

and ϵ is chosen as 10^{-10}. and λ^0 is set as λ_l for PDN defined in Lemma 5, and 0 for SNA. The error is measured by $|\|\mathbf{W}^*\|_{1,\infty} - C|$. Due to the memory limitation, the size of the matrix $\mathbf{A}$ is set as $m = 10000$ and $n = 10000$ and entries of $\mathbf{A}$ are sampled from the normal distribution. We record the error and time (in seconds) of three algorithms, and compute the speedup with respect to SBM. The results are shown in Table 5.1.

The results in Table 5.1 show that our proposed algorithm achieves better performance compared to SNA while maintaining the comparable accuracy of the optimal solution. When the radius becomes small, PDN tends to be even faster.

[1] https://github.com/djchu/projontol1inf

[2] https://www.lsi.upc.edu/~aquattoni/CodeToShare/L1InfProjection.tar.gz

[3] https://goo.gl/TngGrc.

Table 2. $\mathbf{A} \in \mathbb{R}^{m\times n}$ with $m = n = 10000$, averaged in 10 trials

γ	SBM		SNA			PDN		
	ERROR	TIME	ERROR	TIME	SPEEDUP	ERROR	TIME	SPEEDUP
0.01	1.0745E-05	23.781	1.9327E-13	8.1436	2.9202	2.2169E-13	2.8988	**8.2037**
0.05	8.6176E-06	23.775	7.5033E-13	9.5756	2.4829	7.5033E-13	2.9833	**7.9693**
0.1	6.5429E-06	23.785	1.9099E-12	11.151	2.1329	2.5011E-12	3.0049	**7.9153**
0.2	3.3333E-06	23.414	3.0923E-12	11.295	2.073	3.3651E-12	3.1787	**7.3658**
0.3	1.5313E-06	22.197	7.0941E-12	11.034	2.0118	7.8217E-12	3.1281	**7.0962**
0.4	6.3558E-07	22.326	6.9122E-12	12.12	1.8421	5.6389E-12	3.3511	**6.6622**
0.5	2.2989E-07	21.992	7.276E-12	13.511	1.6278	1.7099E-11	3.5405	**6.2116**
0.6	7.2782E-08	22.279	1.4552E-11	14.385	1.5487	1.6735E-11	3.7364	**5.9626**
0.7	1.8139E-08	21.567	2.0009E-11	16.051	1.3436	1.4916E-11	3.9644	**5.44**
0.8	3.0923E-09	22.276	1.819E-11	13.029	1.7097	1.4552E-11	4.3366	**5.1368**
0.9	2.321E-10	21.918	1.7462E-11	8.5278	2.5702	1.9645E-11	4.2581	**5.1474**

We also explore the relationship between the dimension and running time as did in (Chu et al., 2020). We set $m \in \{1000, 2000, \ldots, 10000\}$ and generate the data matrix $\mathbf{A} \in \mathbb{R}^{m\times m}$ from the normal distribution. We set $\gamma = 0.1$, other configurations are the same as the previous. The results are shown in Fig 2. As we can see, as the dimension increases, our algorithm outweighs SBM and SNA in all cases, and the advantage seems significant especially in large-scale cases.

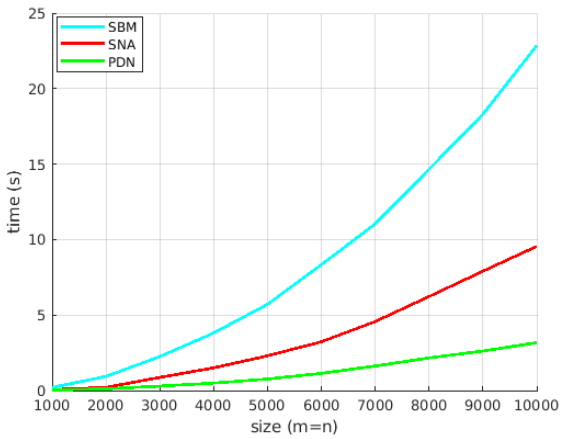

Figure 2. Time versus dimension, SBM (cyan), SNA (red), PDN (green). PDN grows slowest compared to SBM and SNA.

We then investigate the iterate information of PDN, NRFM and SNA. We record the history information of three algorithms for finding the projection of a given matrix $\mathbf{A} \in \mathbb{R}^{1000\times 1000}$ onto the $\ell_{1,\infty}$ norm ball with radius $C = 0.5\|\mathbf{A}\|_{1,\infty}$. SBM is not an iterative algorithm, thus we did not plot its iterate information here. The results are shown in Fig 5.1.

As is can be seen from the Fig 5.1, three algorithms take roughly the same iterations until converges. However, if we

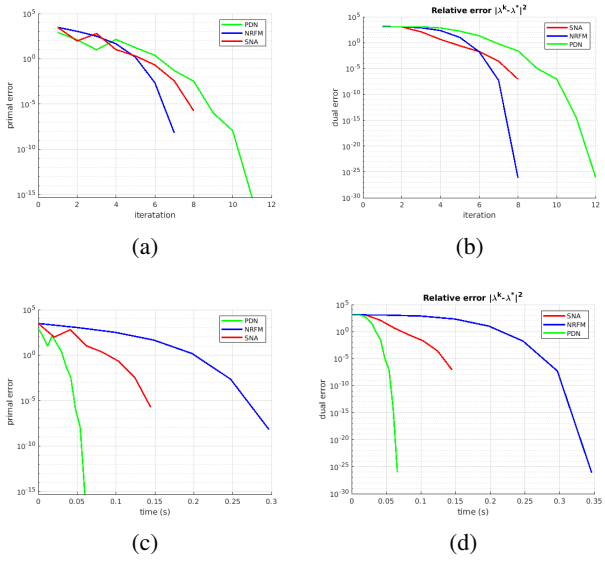

Figure 3. Iteration information of SNA (red), NRFM (blue) and PDN (green). Top: $\|\boldsymbol{\mu}^k - \boldsymbol{\mu}^*\|_2^2$ (a) and $|\lambda^k - \lambda^*|^2$ (b) against iterations. Bottom: $\|\boldsymbol{\mu}^k - \boldsymbol{\mu}^*\|_2^2$ (c) and $|\lambda^k - \lambda^*|^2$ (d) against time.

plot the primal error and dual error againt the time, it can be seen that our proposed algorithm overwhelms NRFM and SNA, this is because our algorithm has lower computational complexity compared with two other algorithms at each iteration, meanwhile it explains the excellent performance of our proposed algorithm. The iterate of λ^k also validates the theoretical results of Lemma 6. That is, the sequence $\{\lambda^k\}$ produced by our algorithm is always an lower bound of that produced by NRFM.

5.2. Multitask Lasso with $\ell_{1,\infty}$ constraint

In this subsection, we explore the performance of our proposed algorithm in the multi-task lasso (Kim et al., 2010)

Table 3. Synthetic data for multitask lasso, M1 (small), M2 (medium), M3 (large), M4(very large).

NAME	(m, d, n)	#NONZEROS	MEMORY
M1	$(10, 5000, 20)$	10^6	7.63MB
M2	$(100, 5000, 20)$	10^7	76.29MB
M3	$(100, 10000, 100)$	10^8	0.75GB
M4	$(800, 15000, 300)$	3.6×10^9	26.822GB

problems, which are typically researched in feature selection. Given data matrices $\mathbf{X}_j \in \mathbb{R}^{n_i \times d}$ of the task j for $j = 1, \ldots, n$. Multi-task Lasso aims to find a matrix $\mathbf{W} = [\boldsymbol{w}_1; \ldots; \boldsymbol{w}_n] \in \mathbb{R}^{d \times n}$, such that the rows of $\mathbf{W}$ measures the importance of the corresponding feature among all tasks. The problem can be written in the form

$$\begin{aligned} \min_{\mathbf{W} \in \mathbb{R}^{d \times n}} \quad & f(\mathbf{W}) := \sum_{i=1}^{n} \frac{1}{2} \|\boldsymbol{y}_i - \mathbf{X}_i \boldsymbol{w}_i\|_2^2 \\ \text{s.t.} \quad & \|\mathbf{W}\|_{1,\infty} \leq C \end{aligned} \tag{25}$$

We use the GPA defined in (2) to solve this problem, the stepsize γ^k is set as $1/\max_i \lambda_{\max}(\mathbf{X}_i \mathbf{X}_i^T)$, where $\lambda_{\max}(\mathbf{A})$ represents the maximum eigenvalue of the matrix $\mathbf{A}$. The projection gradient method is stopped when $\|\mathbf{W}^{k+1} - \mathbf{W}^k\|_F^2 \leq 10^{-5}$ or the maximum iterations (1000 iterations for synthetic data and 10000 iterations for school dataset) is reached, the setup for the projection algorithms are the same as the previous subsection.

5.2.1. SYNTHETIC DATA

We first randomly generate data and compare the performance of three algorithms on the synthetic data. Similar to (Sra, 2011), we configure the number of features d, the number of tasks n and the samples for each task m. The data matrix $\mathbf{X}_i \in \mathbb{R}^{m \times d}$ for $i = 1, \ldots, n$ are then sampled from the normal distribution. The radius is set as $C = 0.1n$. Table 5.2.1 summarizes four different configurations, which varies from small, medium, large, and very large-scale data.

The results are shown in Table 5.2.1. The results show that PDN is around 2 times faster than SNA on large and very large dimension dataset, which illustrates the superior performance of PDN.

5.2.2. REAL-WORLD DATASET

We also test our algorithm on the School dataset[4](Chu et al., 2020). The School dataset consists of $\sum_i n_i = 15362$ test scores of the students from $n = 139$ schools, each student is described with $d = 27$ features. We compare the running time of the projection step in different radius C. The results are shown in Table 5.2.2. Since the dimension of the problem is small, the proposed PDN and SNA are not competitive as the synthetic case. As the dimension of the problem becomes higher, the acceleration effect will become more clear.

[4] https://ttic.uchicago.edu/~argyriou/code

Table 4. multi task lasso with synthetic data. Projection time (s) is recorded.

C/n	SBM	SNA		PDN	
	TIME	TIME	SPEEDUP	TIME	SPEEDUP
M1	26.89	5.35	5.02	3.83	**7.00**
M2	12.63	2.47	5.09	1.86	**6.77**
M3	259.98	47.52	5.47	21.02	**12.36**
M4	1317.8	375.66	3.50	126.58	**10.41**

Table 5. Multi task lasso on School dataset. Projection time (s) is recorded.

C/d	SBM	SNA		PDN	
	TIME	TIME	SPEEDUP	TIME	SPEEDUP
0.01	0.26	0.51	0.52	0.33	0.78
0.05	0.26	0.20	1.28	0.19	**1.34**
0.10	0.26	0.20	1.25	0.19	**1.31**
0.50	0.26	0.20	1.28	0.19	**1.34**
1.00	0.25	0.21	1.22	0.19	**1.32**

6. Conclusion

In this paper, we propose a primal-dual Newton method for efficiently computing the projection onto the $\ell_{1,\infty}$ norm ball. The algorithm alternatively updates the primal and dual variable with the Newton steps, it avoids the computation of the subproblems of projection onto the ℓ_1 ball, which significantly reduces the computational cost. A finite step convergence result is derived. Experiments on synthetic and real-world data exhibits the superior performance of our proposed algorithm especially on large dataset.

References

Alghunaim, S. A. and Sayed, A. H. Linear convergence of primal–dual gradient methods and their performance in distributed optimization. *Automatica*, 117:109003, 2020.

Barzilai, J. and Borwein, J. M. Two-point step size gradient methods. *IMA journal of numerical analysis*, 8(1):141–148, 1988.

Bauschke, H. H. and Combettes, P. L. *Convex Analysis and Monotone Operator Theory in Hilbert Spaces*. Springer

Publishing Company, Incorporated, 1st edition, 2011. ISBN 1441994661.

Beck, A. *First-Order Methods in Optimization*. Society for Industrial and Applied Mathematics, Philadelphia, PA, 2017. doi: 10.1137/1.9781611974997. URL https://epubs.siam.org/doi/abs/10.1137/1.9781611974997.

Béjar, B., Dokmanić, I., and Vidal, R. The fastest $\ell_{1,\infty}$ prox in the west. *arXiv e-prints*, art. arXiv:1910.03749, Oct 2019.

Bertsekas, D. P. On the goldstein-levitin-polyak gradient projection method. *IEEE Transactions on Automatic Control*, 21(2):174–184, 1976.

Birgin, E. G., Martínez, J. M., and Raydan, M. Nonmonotone spectral projected gradient methods on convex sets. *SIAM Journal on Optimization*, 10(4):1196–1211, 2000.

Chau, G., Wohlberg, B., and Rodriguez, P. Efficient projection onto the $\ell_{\infty,1}$ mixed-norm ball using a newton root search method. *SIAM Journal on Imaging Sciences*, 12 (1):604–623, 2019.

Chu, D., Zhang, C., Sun, S., and Tao, Q. Semismooth newton algorithm for efficient projections onto $\ell_{1,\infty}$-norm ball. In *International Conference on Machine Learning*, pp. 1974–1983. PMLR, 2020.

Du, S. S. and Hu, W. Linear convergence of the primal-dual gradient method for convex-concave saddle point problems without strong convexity. In *The 22nd International Conference on Artificial Intelligence and Statistics*, pp. 196–205. PMLR, 2019.

Duchi, J., Shalev-Shwartz, S., Singer, Y., and Chandra, T. Efficient projections onto the ℓ_1-ball for learning in high dimensions. In *Proceedings of the 25th international conference on Machine learning*, pp. 272–279, 2008.

Federer, H. *Geometric measure theory*. Springer, 2014.

Frank, M., Wolfe, P., et al. An algorithm for quadratic programming. *Naval research logistics quarterly*, 3(1-2): 95–110, 1956.

Frasch, J. V., Vukov, M., Ferreau, H. J., and Diehl, M. A dual newton strategy for the efficient solution of sparse quadratic programs arising in sqp-based nonlinear mpc. *Optimization Online 3972*, 2013.

Grippo, L., Lampariello, F., and Lucidi, S. A nonmonotone line search technique for newton's method. *SIAM Journal on Numerical Analysis*, 23(4):707–716, 1986.

Gustavo, C., Wohlberg, B., and Rodriguez, P. Fast projection onto the $\ell_{1,\infty}$-mixed norm ball using steffensen root search. In *2018 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 4694–4698. IEEE, 2018.

Joachim Ferreau, H., Kozma, A., and Diehl, M. A parallel active-set strategy to solve sparse parametric quadratic programs arising in mpc. *IFAC Proceedings Volumes*, 45(17):74–79, 2012. ISSN 1474-6670. doi: https://doi.org/10.3182/20120823-5-NL-3013.00074. URL https://www.sciencedirect.com/science/article/pii/S1474667016314306. 4th IFAC Conference on Nonlinear Model Predictive Control.

Kim, D., Sra, S., and Dhillon, I. S. A scalable trust-region algorithm with application to mixed-norm regression. In *ICML*, pp. 519–526, 2010. URL https://icml.cc/Conferences/2010/papers/562.pdf.

Klintberg, E. and Gros, S. A primal-dual newton method for distributed quadratic programming. In *53rd IEEE Conference on Decision and Control*, pp. 5843–5848, 2014. doi: 10.1109/CDC.2014.7040304.

Kowalski, M. Sparse regression using mixed norms. *Applied and Computational Harmonic Analysis*, 27(3):303 – 324, 2009. ISSN 1063-5203.

Kowalski, M. and Torrésani, B. Sparsity and persistence: mixed norms provide simple signal models with dependent coefficients. *Signal, Image and Video Processing*, 3 (3):251–264, Sep 2009. ISSN 1863-1711.

Liu, H., Palatucci, M., and Zhang, J. Blockwise coordinate descent procedures for the multi-task lasso, with applications to neural semantic basis discovery. In *Proceedings of the 26th Annual International Conference on Machine Learning, ICML'09*, ACM International Conference Proceeding Series, 2009. ISBN 9781605585161.

Luss, R. and Teboulle, M. Conditional gradient algorithms-for rank-one matrix approximations with a sparsity constraint. *SIAM Rev.*, 55(1):65–98, January 2013. ISSN 0036-1445.

Michelot, C. A finite algorithm for finding the projection of a point onto the canonical simplex of $\mathbb{R}^n$. *Journal of Optimization Theory and Applications*, 50(1):195–200, 1986.

Obozinski, G., Taskar, B., and Jordan, M. I. Joint covariate selection and joint subspace selection for multiple classification problems. *Stat. Comput.*, 20(2):231–252, 2010.

Quattoni, A., Carreras, X., Collins, M., and Darrell, T. An efficient projection for $\ell_{1,\infty}$ regularization. *Proceedings of the 26th International Conference On Machine Learning, ICML 2009*, 382, 01 2009. doi: 10.1145/1553374.1553484.

Rockafellar, R. T. *Convex Analysis*. Princeton University Press, 2015. ISBN 9781400873173. doi: doi: 10.1515/9781400873173. URL https://doi.org/10.1515/9781400873173.

Rodriguez, P. An accelerated newton's method for projections onto the $\ell_{1,\infty}$-ball. In *2017 IEEE 27th International Workshop on Machine Learning for Signal Processing (MLSP)*, pp. 1–6, 2017. doi: 10.1109/MLSP.2017.8168161.

Rosen, J. B. The gradient projection method for nonlinear programming. part i. linear constraints. *Journal of the Society for Industrial and Applied Mathematics*, 8(1): 181–217, 1960. ISSN 03684245.

Sra, S. Fast projections onto $\ell_{1,q}$-norm balls for grouped feature selection. In Gunopulos, D., Hofmann, T., Malerba, D., and Vazirgiannis, M. (eds.), *Machine Learning and Knowledge Discovery in Databases*, pp. 305–317, Berlin, Heidelberg, 2011. Springer Berlin Heidelberg. ISBN 978-3-642-23808-6.

Sra, S. Fast projections onto mixed-norm balls with applications. *Data Min. Knowl. Discov.*, 25(2):358–377, September 2012. ISSN 1384-5810.

Tropp, J. A. Algorithms for simultaneous sparse approximation. part ii: Convex relaxation. *Signal Processing*, 86 (3):589–602, 2006.

Wang, L. and Chen, S. Joint representation classification for collective face recognition. *Pattern Recognition*, 63: 182 – 192, 2017. ISSN 0031-3203.